

# Enterprise Post-Call Semantic Evaluation Framework

Using System One / JEV as a semantic judge on top of traditional observability

## Presentation thesis

Traditional observability tells us **what the system did**. JEV evaluates whether the AI **understood the customer, selected the right path, completed the goal, and communicated the outcome correctly**.

| Design principle | Decision                                                                |
|------------------|-------------------------------------------------------------------------|
| Customer path    | Do not put JEV in the normal conversational critical path.              |
| JEV role         | Post-call semantic evaluator / judge.                                   |
| Evidence         | Transcript + structured decisions + tool/API outcomes + final response. |
| System of record | Transactional systems remain authoritative for factual outcomes.        |
| Improvement loop | JEV findings become quality data, regression cases, and tuning inputs.  |

# 1. Master Architecture

The framework separates production execution from semantic evaluation. The customer interaction finishes first; JEV evaluates the interaction asynchronously.

**Customer → Production AI → Call Outcome → Observability**

Production AI: Channel → Conversation Model → Goal Engine → Agent Orchestrator → Specialist Agent → Tools/APIs

Call Outcome: transcript + goals + routing + agent actions + tool results + final response

## Post-call evaluation lane:

**Evidence Sources → Evidence Normalizer → JEV Semantic Judge → JEV Scorecard → Human QA (where needed) → Quality Dataset → Improvement Loop**

| Layer               | Purpose                                | Examples                                         |
|---------------------|----------------------------------------|--------------------------------------------------|
| Production AI       | Serve the customer                     | Conversation model, planner, specialist agents   |
| Observability       | Explain system behavior                | Logs, traces, latency, API status, transfers     |
| Evidence Normalizer | Create a consistent evaluation package | Transcript, goals, routes, tool events, outcomes |
| JEV                 | Judge semantic quality                 | Goal, routing, execution, response, resolution   |
| Quality Dataset     | Turn failures into learning assets     | Labels, failure categories, regression cases     |
| Improvement Loop    | Improve production AI                  | Prompts, NLU, agents, tools, tests               |

## Key architectural rule

JEV is an **inspection lane, not the main highway**. It should normally run after the interaction so semantic evaluation does not add customer-facing voice latency.

## 2. What JEV Evaluates

| Dimension            | Question JEV answers                                | Example failure                                |
|----------------------|-----------------------------------------------------|------------------------------------------------|
| Goal accuracy        | Did the system understand what the customer wanted? | Claim + rental reduced to rental only          |
| Routing accuracy     | Was the customer sent to the right specialist?      | Roadside request routed to an unrelated agent  |
| Goal completion      | Were all requested goals actually completed?        | Claim completed, rental dropped                |
| Agent behavior       | Did the specialist follow the expected workflow?    | Required verification skipped                  |
| Tool/API correctness | Do agent claims match actual system events?         | Agent says success; API failed                 |
| Response accuracy    | Did the final response stay within supported facts? | Invented claim status                          |
| Customer effort      | Did the system create avoidable effort?             | Policy number requested twice                  |
| Resolution           | Was the customer's overall objective resolved?      | Technical success but business outcome partial |
| Policy adherence     | Were required business constraints followed?        | Action taken without required authorization    |

JEV should evaluate the **whole interaction**, not only the last response. A customer can receive a technically correct final sentence while still having an unresolved overall goal.

### 3. Evidence Package Sent to JEV

A transcript alone is insufficient. JEV should receive structured evidence so it can distinguish conversational quality from transactional truth.

```
{ "conversation": { "transcript": "...", "channel": "VOICE" }, "customer_goals": ["FILE_CLAIM", "RENTAL_VEHICLE"], "system_decisions": { "route": "RENTAL_AGENT" }, "tool_events": [ { "tool": "CreateClaim", "status": "SUCCESS"}, { "tool": "ReserveRental", "status": "PENDING"} ], "final_response": "Your claim is filed and your rental is reserved." }
```

Important: **JEV evaluates evidence; the system of record remains authoritative for transactional facts.**

#### Recommended evidence sources

- Conversation transcript and turn-level timestamps
- Detected customer goals and confidence
- Planner / orchestrator decisions
- Agent transfers and specialist selections
- Tool calls, API responses, status codes, and business outcomes
- Customer-provided information captured during the call
- Final response and any commitments made by the AI
- Policy / workflow checkpoints completed or skipped

## 4. Real Insurance Use Case: Accident + Claim + Rental

**Customer:** "I was in an accident. My car isn't drivable. I need to file a claim and I need a rental car."

### Expected goals

- **FILE\_CLAIM** — report the accident / create the claim
- **RENTAL\_VEHICLE** — arrange or initiate rental assistance

### Failure scenario

The conversation model identifies only **RENTAL\_VEHICLE**. The rental workflow succeeds, but the claim workflow is never started.

### JEV finding

```
{ "overall": "FAIL", "severity": "HIGH", "goal_accuracy": 0.42, "routing_accuracy": 0.55,
  "goal_completion": "PARTIAL", "missed_goals": ["FILE_CLAIM"], "root_cause": "Multi-goal
  extraction/routing failure", "recommended_action": "Add labeled example to multi-goal routing
  dataset" }
```

This is exactly where traditional observability can miss the problem: the APIs may all be healthy and the rental transaction may be successful, while the customer's actual objective remains only partially solved.

## 5. Example: Hallucinated Success / Evidence Conflict

**Customer:** "I need to file a claim."

**Agent:** "Your claim has been successfully filed. Your claim number is CLM12345."

**System evidence:** CreateClaim API returned FAILED due to a downstream timeout.

### JEV detects

| Evaluation           | Result                                             |
|----------------------|----------------------------------------------------|
| Agent behavior       | Incorrect                                          |
| Response correctness | Incorrect                                          |
| Evidence consistency | Conflict                                           |
| Severity             | CRITICAL                                           |
| Customer misinformed | Yes                                                |
| Recommended action   | Fix failure-path guardrail and add regression test |

Presentation point: **JEV can identify semantic contradictions between what the AI said and what the system actually did.**

## 6. Example: Unnecessary Customer Effort

The customer provides the policy number during the first 30 seconds. Later, the Claims Agent asks for the same policy number again.

| Signal               | JEV finding                                               |
|----------------------|-----------------------------------------------------------|
| Repeated information | Policy number requested again                             |
| Avoidable turns      | 1 or more                                                 |
| Likely root cause    | Context not propagated to specialist agent                |
| Customer impact      | Increased effort / frustration                            |
| Improvement          | Fix shared conversation state and add regression scenario |

This is a semantic experience problem. Traditional telemetry can show that the second API call succeeded, but it does not inherently understand that the customer had already supplied the information.

## 7. Technical Success ≠ Customer Success

A rental reservation returns HTTP 200. From a technical monitoring perspective, the transaction is successful.

However, the customer originally requested **two goals**: accident reporting and rental assistance. If only the rental succeeds, the overall business outcome is still partial.

| View                     | Result                                                    |
|--------------------------|-----------------------------------------------------------|
| API / infrastructure     | SUCCESS                                                   |
| Rental transaction       | SUCCESS                                                   |
| Claim transaction        | NOT COMPLETED                                             |
| Customer goal completion | 1 of 2                                                    |
| Overall resolution       | PARTIAL                                                   |
| JEV value                | Connects system events to the customer's actual objective |

## 8. Traditional Observability vs Human QA vs JEV

| Capability               | Traditional Observability | Human QA        | JEV                        |
|--------------------------|---------------------------|-----------------|----------------------------|
| Latency / infrastructure | Excellent                 | Low scale       | Not primary purpose        |
| API / tool failures      | Excellent                 | Possible        | Can interpret in context   |
| Goal understanding       | Limited                   | Excellent       | Excellent                  |
| Routing semantics        | Limited                   | Excellent       | Excellent                  |
| Conversation quality     | Limited                   | Excellent       | Scalable                   |
| Customer effort          | Limited                   | Excellent       | Scalable                   |
| Overall resolution       | Limited                   | Excellent       | Scalable                   |
| Scale                    | Very high                 | Low / expensive | High                       |
| Cost                     | Low                       | High            | Moderate / usage-dependent |

### Recommended positioning

**Do not replace observability or human QA.** Use JEV as the scalable semantic evaluation layer between production telemetry and continuous improvement, with humans validating uncertain or high-impact cases.

## 9. Closed-Loop Improvement Framework

**Production Calls → Evidence → JEV → Quality Data → Improvement → Regression Tests → Production**

### Recommended failure categories

- Missed goal
- Wrong route
- Unnecessary transfer
- Hallucinated / unsupported outcome
- Missing context
- Policy deviation
- Unnecessary customer effort
- Tool/API mismatch
- Incomplete resolution

### Every meaningful JEV failure should become one of three assets:

| Asset                  | Purpose                                               |
|------------------------|-------------------------------------------------------|
| Training / tuning data | Improve goal extraction, prompts, routing, and agents |
| Regression scenario    | Prevent the same failure from returning               |
| Operational insight    | Identify systemic issues across journeys or agents    |

## 10. JEV Confidence & Human Calibration

JEV should **not automatically become ground truth** on day one. First establish agreement with human QA.

| JEV confidence | Recommended treatment                                       |
|----------------|-------------------------------------------------------------|
| High           | Potential automated quality labeling, subject to governance |
| Medium         | Sample for human review                                     |
| Low            | Route to human QA / investigation                           |

### Calibration metrics

- JEV vs human QA agreement
- False positives
- False negatives
- Agreement by journey / intent
- Agreement by agent
- Agreement by language and channel
- Severity-weighted disagreement

The goal is not simply to make JEV agree with humans. The goal is to establish where JEV is reliable enough to scale and where human judgment remains necessary.

# 11. Enterprise Quality Dashboard

| Traditional operational metrics | Semantic quality metrics    |
|---------------------------------|-----------------------------|
| Call completion                 | Goal accuracy               |
| Average handle time             | Routing accuracy            |
| API success rate                | Goal completion             |
| Latency                         | Customer effort             |
| Transfer rate                   | Response correctness        |
| Error rate                      | Unnecessary transfer        |
| Tool failures                   | Agent quality               |
| System availability             | Overall customer resolution |

A combined dashboard gives leadership both views: **Is the platform healthy?** and **Is the AI actually solving customer problems?**

## 12. Recommended POC

**Scope:** one high-value journey — Auto Accident → Claim + Rental.

| Phase                | Activity                                                 | Outcome                         |
|----------------------|----------------------------------------------------------|---------------------------------|
| 1. Baseline          | Collect transcript, routing, agent actions, API outcomes | Evaluation-ready evidence       |
| 2. JEV evaluation    | Run semantic judge over historical calls                 | Initial quality scorecard       |
| 3. Human calibration | Compare JEV against QA samples                           | Confidence / agreement baseline |
| 4. Improvement       | Convert failures into tuning and regression data         | Closed-loop improvement         |
| 5. Scale             | Extend to additional journeys and agents                 | Enterprise semantic evaluation  |

Post-call processing can be asynchronous using an event-driven pipeline, so the JEV evaluation does not block the customer's voice interaction.

## 13. Suggested Enterprise Implementation

| Component                 | Responsibility                                        |
|---------------------------|-------------------------------------------------------|
| Conversation platform     | Produces transcript and interaction events            |
| Event / queue layer       | Decouples production traffic from evaluation          |
| Evidence normalizer       | Combines transcript, decisions, tools, APIs, outcomes |
| JEV adapter               | Invokes JEV with versioned evaluation contract        |
| Evaluation store          | Persists scorecards, findings, evidence references    |
| Dashboard                 | Operational + semantic quality views                  |
| Human QA workflow         | Reviews sampled / low-confidence cases                |
| Quality dataset           | Stores labeled failure examples and gold cases        |
| Model / agent improvement | Feeds prompts, classifiers, workflows, tests          |

### Recommended governance fields

- Evaluator version
- Prompt / rubric version
- Journey / agent version
- Timestamp
- JEV confidence
- Human review status
- Severity
- Root-cause category

## 14. Final Architecture Principle

The strongest use of JEV is not to make every customer turn slower. It is to make the enterprise AI system **measurably better after every interaction**.

**Production AI executes.**

**Traditional observability explains what happened.**

**JEV judges whether it was actually good.**

**Human QA calibrates the judge.**

**The improvement loop makes the next interaction better.**

### Presentation takeaway

“We don't put another model in the customer's critical path. We use JEV after the interaction to understand whether our AI actually solved the customer's problem — and turn those findings into a continuous improvement loop.”

Draft — company-neutral architecture